MCP en pratique : serveur Python

Écrire un serveur avec le SDK officiel `mcp` 2.x (MCPServer), le lancer en stdio, le tester avec MCP Inspector

bstorm TRAINING

MCP en bref

Model Context Protocol : protocole JSON-RPC qui relie une application d'IA à des outils et des données. Un serveur écrit une fois sert tous les clients compatibles (Claude Desktop, Claude Code, Agent SDK, autres).

Rôle	Exemple
Hôte : application qui porte le modèle	Claude Desktop, script Agent SDK
Client : connexion de l'hôte vers un serveur	un client par serveur, créé par l'hôte
Serveur : processus qui expose des capacités	`serveur_notes.py`

Primitive	Décorateur	Qui l'utilise
Outil (tool) : action avec arguments	`@mcp.tool()`	le modèle décide de l'appeler
Ressource : donnée lisible par URI	`@mcp.resource("notes://{nom}")`	l'application ou l'utilisateur la joint au contexte
Prompt : modèle de consigne	`@mcp.prompt()`	l'utilisateur le choisit

Transports : **stdio** (le client lance le serveur en sous-processus, échanges sur stdin/stdout, usage local) ; **Streamable HTTP** (serveur distant ou partagé, URL `http://hote:port/mcp`). Spécification 2026-07-28 : protocole sans état, plus d'en-tête de session `Mcp-Session-Id`.

Installer

```
py -3.12 -m venv .venv
# [cli] ajoute la commande mcp
.\.venv\Scripts\python.exe -m pip install "mcp[cli]"
.\.venv\Scripts\mcp.exe version          # MCP version 2.x
```

> **Version 2 du SDK.** `FastMCP` est renommé `MCPServer`. `from mcp.server.fastmcp import FastMCP` lève `ModuleNotFoundError` en 2.x. Ancien code v1 : adapter l'import, ou épingler `pip install "mcp<2"`. Les décorateurs `@mcp.tool()`, `@mcp.resource()`, `@mcp.prompt()` sont inchangés.

Serveur d'exemple : `serveur_notes.py`

```
"""Serveur MCP d'exemple : notes texte dans un dossier (stdio)."""
import sys
from pathlib import Path

from mcp.server import MCPServer
from mcp.server.mcpserver.exceptions import ToolError
from mcp.types import ToolAnnotations

DOSSIER = Path(__file__).parent / "notes"    # absolu, indépendant du cwd
DOSSIER.mkdir(exist_ok=True)
LECTURE = ToolAnnotations(read_only_hint=True)

mcp = MCPServer("notes", instructions="Notes texte locales.")

@mcp.tool(annotations=LECTURE)
def lister_notes() -> list[str]:
    """Liste les noms des notes disponibles."""
    return sorted(p.stem for p in DOSSIER.glob("*.txt"))

@mcp.tool(annotations=LECTURE)
def lire_note(nom: str) -> str:
    """Renvoie le contenu de la note `nom` (sans extension)."""
    fichier = DOSSIER / f"{Path(nom).name}.txt"  # .name bloque ../
    if not fichier.exists():
        raise ToolError(f"Note inconnue : {nom}")
    return fichier.read_text(encoding="utf-8")

@mcp.tool()
def ajouter_note(nom: str, texte: str) -> str:
    """Crée ou remplace la note `nom`."""
    (DOSSIER / f"{Path(nom).name}.txt").write_text(texte, encoding="utf-8")
    print(f"note {nom} ecrite", file=sys.stderr)  # journal : stderr
    return f"Note {nom} enregistrée."

@mcp.resource("notes://{nom}")
def note(nom: str) -> str:
    """Note exposée comme ressource."""
    return lire_note(nom)

@mcp.prompt()
def resumer(nom: str) -> str:
    """Prompt prêt à l'emploi : résumer une note."""
    return f"Lis la note {nom} avec lire_note puis résume-la en 3 points."

if __name__ == "__main__":
    mcp.run()          # transport stdio par défaut
```

Règles d'écriture

- **Docstring = description** envoyée au modèle : dire quoi, quand l'utiliser, format des arguments. Le modèle choisit l'outil sur ce texte.
- **Annotations de type = schéma** : `nom: str` donne `inputSchema` ; le type de retour donne `outputSchema` et un résultat structuré (`structuredContent`, ex. `{"result": ["essai"]}`).
- **Erreur métier** : `raise ToolError("...")`. Le client reçoit un résultat `is_error = True` avec le message ; le modèle peut corriger et réessayer.
- **stdout réservé au protocole** en stdio : tout `print()` sur stdout corrompt les échanges. Journaliser sur `sys.stderr` ou dans un fichier.
- **Chemins absolus** : le client lance le serveur depuis un dossier quelconque. Construire les chemins depuis `Path(__file__).parent`.
- **Annotations d'outil** : `ToolAnnotations(read_only_hint=True, destructive_hint=False, idempotent_hint=True)` informent le client (affichage, demande de confirmation). Indication seulement : la protection réelle reste dans le code (ex. base SQLite ouverte en `mode=ro`).
- **Peu d'outils, bien nommés**, arguments simples ; limiter la taille des retours (pagination, nombre maximal de lignes).
- **Contexte de requête** : ajouter un paramètre `ctx: Context` (`from mcp.server.mcpserver import Context`) pour `await ctx.report_progress(1, 2)`. Fonction `async def` si elle utilise la boucle asyncio.

Lancer en stdio

```
# direct : le serveur attend du JSON-RPC sur stdin, rien ne s'affiche
.\.venv\Scripts\python.exe serveur_notes.py          # Ctrl+C pour quitter
# via la CLI du SDK
.\.venv\Scripts\mcp.exe run serveur_notes.py
```

En pratique, le serveur n'est pas lancé à la main : le client (Inspector, Claude Desktop, Agent SDK) le démarre avec la commande déclarée. Variante HTTP : `mcp.run(transport="streamable-http", host="127.0.0.1", port=8000)`, URL `http://127.0.0.1:8000/mcp`.

Tester avec MCP Inspector

Outil de test du projet MCP (Node.js requis, `node --version`).

```
# interface web : http://127.0.0.1:6274 (jeton dans l'URL affichée)
npx -y @modelcontextprotocol/inspector `
    .\.venv\Scripts\python.exe serveur_notes.py
```

1. Onglet **Servers** : activer l'interrupteur du serveur (statut **Connected**).
2. Onglet **Tools** : la liste se charge seule ; choisir `ajouter_note`, remplir le formulaire, **Execute Tool**. Résultat dans **Results** (**Tool Error** si `ToolError`).
3. Onglets **Resources** (`notes://essai`) et **Prompts** (`resumer`).
4. Panneau latéral : **Protocol** (messages JSON-RPC échangés), **Console** (stderr du serveur).

```
# mode ligne de commande, sans navigateur (scriptable ; --format json)
npx -y @modelcontextprotocol/inspector --cli `
    .\.venv\Scripts\python.exe serveur_notes.py --method tools/list
npx -y @modelcontextprotocol/inspector --cli `
    .\.venv\Scripts\python.exe serveur_notes.py `
    --method tools/call --tool-name lire_note --tool-arg nom=essai
```

`mcp dev serveur_notes.py` lance aussi l'Inspector, mais passe par `uv` (à installer à part).

Tester en Python : `client_test.py`

```
import asyncio, sys
from pathlib import Path
from mcp import Client, StdioServerParameters

SERVEUR = Path(__file__).with_name("serveur_notes.py")

async def main():
    params = StdioServerParameters(command=sys.executable,
                                   args=[str(SERVEUR)])
    async with Client(params) as client:          # lance le serveur
        print(client.protocol_version)            # 2026-07-28
        outils = await client.list_tools()
        print([t.name for t in outils.tools])
        r = await client.call_tool("ajouter_note",
                                   {"nom": "essai", "texte": "bonjour"})
        r = await client.call_tool("lister_notes", {})
        print(r.structured_content)               # {'result': ['essai']}
        res = await client.read_resource("notes://essai")
        print(res.contents[0].text)               # bonjour

asyncio.run(main())
```

MCP en pratique : brancher le serveur

Claude Desktop (Windows), Agent SDK Python, claude -p, Claude Code · dépannage

Claude Desktop

Fichier de configuration

Système	Chemin
Windows	`%APPDATA%\Claude\claude_desktop_config.json` soit `C:\Users\<utilisateur>\AppData\Roaming\Claude\`
macOS	`~/Library/Application Support/Claude/claude_desktop_config.json`

Installation par le Microsoft Store (MSIX) : le fichier lu est sous `%LOCALAPPDATA%\Packages\Claude_*\LocalCache\Roaming\Claude\`, pas sous `%APPDATA%`.

Accès depuis l'application : **Paramètres > Développeur > Modifier la configuration** (crée le fichier s'il n'existe pas). Depuis PowerShell :

```
notepad "$env:APPDATA\Claude\claude_desktop_config.json"
```

Déclarer le serveur

```
{
  "mcpServers": {
    "notes": {
      "command": "C:\\agents\\.venv\\Scripts\\python.exe",
      "args": ["C:\\agents\\serveur_notes.py"]
    }
  }
}
```

- **Chemins absolus** partout, antislash doublé en JSON (`\\`) ou barres obliques (`C:/agents/...`).
- **Python du venv** dans `command` : il contient le paquet `mcp`. Un `python` nu prend l'interpréteur global, sans `mcp`.
- Conserver les autres clés du fichier ; ajouter le serveur dans `mcpServers`. JSON invalide = aucun serveur chargé.
- Clé facultative `"env": {"NOM": "valeur"}` : variables passées au serveur (chemin de base, jeton d'un service).

Recharger et vérifier

1. Quitter complètement Claude Desktop (icône de la zone de notification > Quitter ; fermer la fenêtre ne suffit pas), puis relancer. L'application propose aussi **Paramètres > Développeur > Recharger la configuration MCP**.
2. Dans la zone de saisie : bouton **+** > **Connecteurs** : le serveur `notes` apparaît avec ses outils, activables un par un.
3. Demander « Liste mes notes » : Claude demande l'autorisation avant chaque appel d'outil.

Journaux (Windows)

```
Get-ChildItem "$env:APPDATA\Claude\logs"
Get-Content "$env:APPDATA\Claude\logs\mcp-server-notes.log" -Tail 30 -Wait
```

`mcp.log` : connexions et échecs ; `mcp-server-<nom>.log` : stderr du serveur.

Alternative : `mcp install serveur_notes.py --name notes` écrit l'entrée automatiquement, mais lance le serveur via `uv run` (uv requis).

Claude Code et `claude -p`

```
# Claude Code : enregistrer le serveur (portée locale par défaut)
claude mcp add notes -- C:\agents\.venv\Scripts\python.exe `
    C:\agents\serveur_notes.py
claude mcp list
```

`mcp.json` pour un run ponctuel (même format que Claude Desktop) :

```
& $claude -p "Liste mes notes" --mcp-config mcp.json `
    --strict-mcp-config --allowedTools "mcp__notes__*"
```

`--strict-mcp-config` : seuls les serveurs de `--mcp-config` sont chargés. `$claude` : voir fiche « composants et usage en local ».

Agent SDK Python

serveur stdio externe

```
import asyncio, sys
from pathlib import Path
from claude_agent_sdk import (AssistantMessage, ClaudeAgentOptions,
    ResultMessage, SystemMessage, ToolUseBlock, query)

SERVEUR = Path(__file__).with_name("serveur_notes.py")

options = ClaudeAgentOptions(
    model="claude-haiku-5-5",
    system_prompt="Tu gères des notes. Réponds brièvement.",
    tools=[],                                  # pas d'outil intégré
    mcp_servers={"notes": {
        "type": "stdio",
        "command": sys.executable,             # python du venv
        "args": [str(SERVEUR)],
    }},
    allowed_tools=["mcp__notes__lister_notes",
                   "mcp__notes__lire_note"],   # pas ajouter_note
    max_turns=5, setting_sources=[])

async def main():
    async for m in query(prompt="Résume la note essai",
                         options=options):
        if isinstance(m, SystemMessage) and m.subtype == "init":
            # statut : connected, pending, failed, needs-auth, disabled
            print(m.data.get("mcp_servers"))
        elif isinstance(m, AssistantMessage):
            for b in m.content:
                if isinstance(b, ToolUseBlock):
                    print("outil :", b.name, b.input)
        elif isinstance(m, ResultMessage):
            print(m.result)

asyncio.run(main())
```

- Nom vu par le modèle : `mcp__<serveur>__<outil>`. Joker : `"mcp__notes__*"`.
- Un outil MCP absent de `allowed_tools` est visible mais son appel n'est pas exécuté sans autorisation. `permission_mode="acceptEdits"` n'autorise pas les outils MCP.
- Autres formes : `{"type": "http", "url": "https://.../mcp", "headers": {...}}` ; fichier `.mcp.json` du projet, chargé si `setting_sources` contient `"project"`.
- Outils écrits dans le script même, sans processus séparé : `@tool` + `create_sdk_mcp_server()` (fiche « composants et usage en local »).
- Mode abonnement : connexion préalable `claude.exe auth login --claudeai`, aucune variable `ANTHROPIC_API_KEY`.

Dépannage

Symptôme	Cause probable, correction
Serveur absent de Claude Desktop	JSON invalide, application non quittée complètement, chemin relatif. Lire `mcp.log`.
`ModuleNotFoundError: mcp` dans le log	`command` pointe vers un Python sans `mcp` : mettre le `python.exe` du venv.
`ModuleNotFoundError: mcp.server.fastmcp`	Code v1 sur SDK 2.x : `from mcp.server import MCPServer`.
Connexion qui échoue sans message clair	`print()` sur stdout : passer sur `sys.stderr`.
Fichier introuvable dans un outil	Chemin relatif au cwd : partir de `Path(__file__).parent`.
Outil listé mais jamais appelé	Docstring floue, ou outil non autorisé (`allowed_tools`, `--allowedTools`, réglage du connecteur).
Statut `failed` à l'init (Agent SDK)	Tester d'abord la même commande dans l'Inspector ; délai de démarrage : variable `MCP_TIMEOUT` (ms, 30 s par défaut).
Résultat d'outil tronqué ou déporté en fichier	Retour trop gros (plus de 25 000 tokens) : paginer, filtrer côté serveur.

Ordre de travail

1. Écrire le serveur, 1 à 3 outils, docstrings précises.
2. `client_test.py` ou Inspector `--cli` : chaque outil répond, les erreurs passent par `ToolError`.
3. Inspector web : relire descriptions et schémas tels que le modèle les verra.
4. Brancher dans Claude Desktop ou l'Agent SDK, avec la liste d'outils autorisés la plus courte possible.
5. Lire les journaux après les premiers runs.